

Relazione tecnica di sicurezza per applicazione React + Express

1. Introduzione

Una web application moderna basata su React ed Express e' tipicamente organizzata secondo un'architettura client-server distribuita. Il frontend e' una Single Page Application (SPA) sviluppata in React, eseguita nel browser dell'utente. Il backend e' un'applicazione Node.js con framework Express.js, responsabile dell'esposizione di API REST, della logica applicativa, dell'accesso ai dati, dell'autenticazione e dell'applicazione delle policy di sicurezza.

Nel modello SPA, il browser scarica inizialmente file statici come HTML, JavaScript, CSS e asset. Dopo il caricamento iniziale, la navigazione avviene principalmente lato client: React aggiorna la UI senza ricaricare l'intera pagina. La comunicazione con il server avviene tramite richieste HTTP/HTTPS verso endpoint REST, ad esempio `GET /api/profile`, `POST /api/login`, `PUT /api/settings` o `DELETE /api/items/:id`. Le risposte sono di norma in formato JSON.

Il flusso tipico e' il seguente:

1. L'utente apre l'applicazione nel browser tramite HTTPS.
2. Il server o un servizio di hosting statico consegna la SPA React.
3. React effettua chiamate alle API REST usando `fetch` o librerie come Axios.
4. Express riceve le richieste, valida input, autorizza l'operazione e interagisce con database o servizi esterni.
5. Il backend restituisce dati JSON o errori controllati.
6. Il browser applica le regole di sicurezza del contesto web: Same-Origin Policy, CORS, cookie policy, Content Security Policy e controlli TLS.

In questa architettura possono essere usati cookie di sessione, token JWT o una combinazione di entrambi. I cookie di sessione sono spesso associati a sessioni server-side e vengono inviati automaticamente dal browser a ogni richiesta verso il dominio applicativo. I token JWT sono invece credenziali firmate, spesso inviate tramite header `Authorization: Bearer`, e richiedono particolare attenzione nella gestione, nella durata e nel luogo di memorizzazione lato client.

I concetti base di sicurezza web includono:

- **Riservatezza:** i dati devono essere leggibili solo da soggetti autorizzati.
- **Integrita':** i dati non devono essere alterati in modo non autorizzato durante trasmissione, elaborazione o persistenza.
- **Disponibilita':** il servizio deve rimanere accessibile agli utenti legittimi.
- **Autenticazione:** verifica dell'identita' dell'utente o del client.
- **Autorizzazione:** verifica dei permessi associati all'identita' autenticata.
- **Non ripudio e tracciabilita':** conservazione di log coerenti per audit e analisi forense.
- **Defense in depth:** adozione di controlli multipli a diversi livelli, dal browser alla rete, dal backend al database.

La sicurezza e' particolarmente importante nelle applicazioni distribuite perche' la superficie di attacco non coincide con un singolo componente. Una vulnerabilita' puo' nascere nel codice React, nella configurazione Express, nella gestione dei cookie, nella logica di autorizzazione, nel database, nella terminazione TLS o in un proxy intermedio. Per questo motivo la sicurezza deve essere progettata in modo multilivello, considerando applicazione, browser, protocollo HTTP, TLS, rete locale, DNS e infrastruttura di deployment.

2. Elenco delle vulnerabilita'

Le vulnerabilita' analizzate in questa relazione sono:

- **Cross-Site Scripting (XSS)**: esecuzione di codice JavaScript non autorizzato nel browser dell'utente a causa di output non correttamente neutralizzato.
- **Cross-Site Request Forgery (CSRF)**: induzione del browser autenticato dell'utente a inviare richieste indesiderate verso l'applicazione.
- **SQL Injection**: inserimento di input malevolo in query SQL costruite dinamicamente senza parametrizzazione.
- **Furto e manipolazione dei cookie**: esposizione o modifica di cookie usati per autenticazione, preferenze o autorizzazione.
- **Session Hijacking**: uso non autorizzato di una sessione valida, ad esempio tramite furto di identificativi di sessione o token.
- **Clickjacking**: incorporamento dell'applicazione in frame controllati da terzi per indurre l'utente a interagire con elementi non percepiti correttamente.
- **Man-in-the-Middle (MITM)**: intercettazione o alterazione della comunicazione tra client e server quando il trasporto non e' protetto o e' configurato male.
- **Attacchi a livello rete, DNS spoofing e ARP spoofing**: manipolazioni concettuali della risoluzione dei nomi o dell'instradamento locale per deviare il traffico.
- **Vulnerabilita' HTTP/TLS e mancanza di header di sicurezza**: assenza di policy browser-side e configurazioni di trasporto insufficienti.

3. Sezioni dedicate a ogni vulnerabilita'

● Cross-Site Scripting (XSS)

1. Descrizione tecnica

Il Cross-Site Scripting e' una vulnerabilita' che consente l'esecuzione di JavaScript non autorizzato nel contesto di sicurezza dell'applicazione. Il problema si verifica quando dati controllabili dall'utente vengono inseriti nel DOM o restituiti al browser senza adeguata codifica, sanitizzazione o separazione tra dati e codice.

Le forme principali sono:

- **Stored XSS**: il contenuto pericoloso viene salvato nel database e servito ad altri utenti.
- **Reflected XSS**: il contenuto arriva nella richiesta e viene riflesso immediatamente nella risposta.
- **DOM-based XSS**: la manipolazione insicura avviene interamente lato browser, tramite API DOM o routing client-side.

React riduce il rischio perche' applica escaping automatico dei valori interpolati nel JSX. Tuttavia, il rischio rimane quando si usa `dangerouslySetInnerHTML`, quando si manipola direttamente il DOM, quando si renderizza HTML proveniente da utenti o CMS, oppure quando il backend restituisce contenuti HTML non affidabili.

2. Come si manifesta in React + Express

In una SPA React + Express, XSS puo' manifestarsi quando:

- Express salva o restituisce commenti, descrizioni o profili utente contenenti HTML non validato.
- React inserisce contenuti ricevuti dall'API con `dangerouslySetInnerHTML`.
- Il frontend costruisce manualmente markup HTML partendo da input esterno.
- La Content Security Policy e' assente o troppo permissiva.
- Cookie o token accessibili a JavaScript vengono letti da codice eseguito nel contesto della pagina.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";

const app = express();
app.use(express.json());

const comments = [];

app.post("/api/comments", (req, res) => {
  const comment = {
    author: req.body.author,
    body: req.body.body
  };

  comments.push(comment);
  res.status(201).json(comment);
});

app.get("/api/comments", (req, res) => {
  res.json(comments);
});
```

Frontend vulnerabile (React):

```
import { useEffect, useState } from "react";

export function Comments() {
  const [comments, setComments] = useState([]);

  useEffect(() => {
    fetch("/api/comments")
      .then((res) => res.json())
      .then(setComments);
  }, []);

  return (
    <section>
      {comments.map((comment, index) => (
        <article key={index}>
          <h3>{comment.author}</h3>
          <div dangerouslySetInnerHTML={{ __html: comment.body }} />
        </article>
      ))}
    </section>
  );
}
```

Il problema non e' la presenza di React, ma l'uso di un'API che disabilita consapevolmente l'escaping automatico.

4. Scenario di attacco (solo descrizione teorica, non operativa)

Un utente inserisce in un campo commento un contenuto interpretato dal browser come codice. Il backend lo memorizza senza validazione. Quando altri utenti aprono la pagina dei commenti, il frontend renderizza il contenuto come HTML e il browser esegue codice nel contesto dell'applicazione. In termini teorici, questo potrebbe permettere lettura di dati visibili nella pagina, azioni a nome dell'utente o abuso di token accessibili via JavaScript.

5. Impatto sulla sicurezza del sistema

L'impatto puo' essere elevato:

- Compromissione della sessione utente se token o dati sensibili sono accessibili al codice client.
- Azioni non autorizzate nel contesto dell'utente autenticato.
- Alterazione della UI e phishing interno all'applicazione.
- Esfiltrazione di dati mostrati nella SPA.
- Propagazione del problema ad altri utenti nel caso di stored XSS.

6. Versione corretta del codice (mitigata)

Backend sicuro:

```
import express from "express";
import helmet from "helmet";
import { z } from "zod";

const app = express();
app.use(express.json());

app.use(
  helmet({
    contentSecurityPolicy: {
      useDefaults: true,
      directives: {
        "default-src": ["'self'"],
        "script-src": ["'self'"],
        "object-src": ["'none'"],
        "base-uri": ["'self'"],
        "frame-ancestors": ["'none'"]
      }
    }
  })
);

const commentSchema = z.object({
  author: z.string().trim().min(1).max(80),
  body: z.string().trim().min(1).max(2000)
});

const comments = [];

app.post("/api/comments", (req, res) => {
  const parsed = commentSchema.safeParse(req.body);

  if (!parsed.success) {
    return res.status(400).json({ error: "Invalid comment payload" });
  }

  const comment = {
    author: parsed.data.author,
    body: parsed.data.body
  };

  comments.push(comment);
  return res.status(201).json(comment);
});

app.get("/api/comments", (req, res) => {
  return res.json(comments);
});
```

Frontend sicuro:

```
import { useEffect, useState } from "react";

export function Comments() {
  const [comments, setComments] = useState([]);

  useEffect(() => {
    fetch("/api/comments", { credentials: "include" })
      .then((res) => {
        if (!res.ok) throw new Error("Unable to load comments");
        return res.json();
      })
      .then(setComments);
  }, []);

  return (
    <section>
      {comments.map((comment, index) => (
        <article key={index}>
          <h3>{comment.author}</h3>
          <p>{comment.body}</p>
        </article>
      ))}
    </section>
  );
}
```

7. Mitigazioni

Lato backend:

- Validare input con librerie come `zod`, `joi` o `express-validator`.
- Restituire dati come JSON, non come HTML costruito da input utente.
- Applicare `helmet` con una Content Security Policy restrittiva.
- Normalizzare lunghezze, tipi e formato dei campi ricevuti.
- Salvare nel database dati semanticamente validi, evitando di trattare input utente come markup eseguibile.

Lato frontend:

- Affidarsi all'escaping automatico di React tramite interpolazione JSX ordinaria.
- Evitare `dangerouslySetInnerHTML`; se indispensabile, usare una libreria di sanitizzazione robusta e una allowlist molto limitata.
- Non memorizzare token sensibili in `localStorage` quando non necessario.
- Evitare manipolazioni DOM manuali con contenuto non affidabile.

Lato rete:

- Usare sempre HTTPS per impedire l'iniezione di script durante il transito.
- Applicare HSTS per ridurre il rischio di downgrade verso HTTP.

- Usare cookie **HttpOnly**, **Secure** e **SameSite** per limitare gli effetti di eventuale codice eseguito nel browser.

● Cross-Site Request Forgery (CSRF)

1. Descrizione tecnica

Il Cross-Site Request Forgery sfrutta il comportamento automatico del browser nell'inviare cookie associati a un dominio. Se un utente e' autenticato su un'applicazione che usa cookie di sessione, una pagina esterna potrebbe indurre il browser a inviare una richiesta verso l'applicazione bersaglio. Se il server controlla solo la presenza del cookie, ma non verifica l'intenzione dell'utente, l'operazione puo' essere accettata.

Il rischio riguarda soprattutto richieste che modificano stato: cambio email, cambio password, trasferimenti, acquisti, cancellazioni o modifiche di configurazione. Le API REST non sono automaticamente immuni: se sono accessibili con cookie inviati dal browser, devono prevedere protezioni anti-CSRF.

2. Come si manifesta in React + Express

In React + Express, CSRF puo' manifestarsi quando:

- L'autenticazione usa cookie di sessione.
- Gli endpoint **POST**, **PUT**, **PATCH** o **DELETE** non richiedono token anti-CSRF.
- I cookie sono configurati con **SameSite=None** senza una ragione stretta.
- CORS e cookie cross-site sono configurati in modo eccessivamente permissivo.
- Il backend non verifica header di origine come **Origin** e **Referer** dove appropriato.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";
import session from "express-session";

const app = express();
app.use(express.json());

app.use(
  session({
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      secure: true,
      sameSite: "none"
    }
  })
);

app.post("/api/account/email", (req, res) => {
  if (!req.session.userId) {
    return res.status(401).json({ error: "Unauthorized" });
  }

  updateEmail(req.session.userId, req.body.email);
  return res.json({ ok: true });
});
```

Frontend vulnerabile (React):

```
async function changeEmail(email) {
  await fetch("/api/account/email", {
    method: "POST",
    credentials: "include",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email })
  });
}
```

La richiesta e' autenticata solo grazie al cookie. Non esiste un valore specifico della sessione che provi che la richiesta sia partita dalla UI legittima.

4. Scenario di attacco (solo descrizione teorica, non operativa)

Un utente autenticato visita una pagina esterna. Tale pagina induce il browser dell'utente a inviare una richiesta verso l'applicazione vulnerabile. Poiche' il browser allega automaticamente il cookie di sessione, il server interpreta la richiesta come autenticata e modifica dati dell'account. La descrizione e' concettuale: la

prevenzione si basa sul distinguere una richiesta intenzionale generata dall'applicazione da una richiesta cross-site non autorizzata.

5. **Impatto sulla sicurezza del sistema**

L'impatto dipende dalle operazioni esposte:

- Modifica di dati personali.
- Cambio email o impostazioni di sicurezza.
- Esecuzione di operazioni amministrative se l'utente ha privilegi elevati.
- Cancellazione o modifica di risorse.
- Violazione dell'integrità applicativa.

6. **Versione corretta del codice (mitigata)**

Backend sicuro:

```
import crypto from "node:crypto";
import express from "express";
import session from "express-session";
import { z } from "zod";

const app = express();
app.set("trust proxy", 1);
app.use(express.json());

app.use(
  session({
    name: "__Host-sid",
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      secure: true,
      sameSite: "lax",
      path: "/",
      maxAge: 1000 * 60 * 60
    }
  })
);

function requireAuth(req, res, next) {
  if (!req.session.userId) {
    return res.status(401).json({ error: "Unauthorized" });
  }
  return next();
}

app.get("/api/csrf-token", requireAuth, (req, res) => {
  req.session.csrfToken = crypto.randomBytes(32).toString("hex");
  return res.json({ csrfToken: req.session.csrfToken });
});

function requireCsrf(req, res, next) {
  const sentToken = req.get("X-CSRF-Token");

  if (!req.session.csrfToken || sentToken !== req.session.csrfToken) {
    return res.status(403).json({ error: "Invalid CSRF token" });
  }

  return next();
}

const emailSchema = z.object({
  email: z.string().trim().email().max(254)
});

app.post("/api/account/email", requireAuth, requireCsrf, (req, res) => {
```

```
const parsed = emailSchema.safeParse(req.body);

if (!parsed.success) {
  return res.status(400).json({ error: "Invalid email" });
}

updateEmail(req.session.userId, parsed.data.email);
return res.json({ ok: true });
});
```

Frontend sicuro:

```
import { useEffect, useState } from "react";

export function EmailSettings() {
  const [csrfToken, setCsrfToken] = useState("");

  useEffect(() => {
    fetch("/api/csrf-token", { credentials: "include" })
      .then((res) => res.json())
      .then((data) => setCsrfToken(data.csrfToken));
  }, []);

  async function changeEmail(email) {
    const response = await fetch("/api/account/email", {
      method: "POST",
      credentials: "include",
      headers: {
        "Content-Type": "application/json",
        "X-CSRF-Token": csrfToken
      },
      body: JSON.stringify({ email })
    });

    if (!response.ok) {
      throw new Error("Email update failed");
    }
  }

  return null;
}
```

7. Mitigazioni

Lato backend:

- Usare token anti-CSRF associati alla sessione per richieste state-changing.
- Configurare cookie `SameSite=Lax` o `SameSite=Strict` quando possibile.
- Verificare `Origin` e `Referer` per richieste sensibili, come controllo aggiuntivo.
- Limitare CORS a origini esplicitamente autorizzate.
- Separare endpoint pubblici da endpoint autenticati e applicare middleware coerenti.

Lato frontend:

- Inviare token anti-CSRF tramite header custom, non tramite parametri prevedibili.
- Usare `credentials: "include"` solo quando necessario.
- Evitare form o chiamate automatiche non intenzionali su operazioni distruttive.
- Gestire errori 401 e 403 in modo chiaro senza ripetere richieste all'infinito.

Lato rete:

- Usare HTTPS per proteggere cookie e token anti-CSRF in transito.

- Impostare cookie **Secure** affinché non siano inviati su HTTP.
- Usare HSTS per rendere persistente l'uso di HTTPS lato browser.

● SQL Injection

1. Descrizione tecnica

La SQL Injection si verifica quando input controllabile dall'utente viene concatenato direttamente in una query SQL. Il database interpreta parte dell'input come sintassi SQL invece che come valore. Il difetto nasce dalla mancata separazione tra codice SQL e dati.

La mitigazione fondamentale e' l'uso di query parametrizzate, prepared statements o ORM che generano query sicure. La validazione dell'input e' necessaria ma non sostituisce la parametrizzazione, perche' la protezione principale deve avvenire al livello dell'interfaccia con il database.

2. Come si manifesta in React + Express

In React + Express, SQL Injection puo' manifestarsi quando:

- Il backend costruisce query con template string o concatenazione.
- Parametri di rotta, query string o body JSON vengono usati direttamente in SQL.
- Il frontend consente filtri, ricerca o ordinamenti che vengono convertiti in frammenti SQL non validati.
- L'applicazione espone messaggi di errore SQL dettagliati al client.

React non interagisce direttamente con il database, ma puo' inviare input che il backend gestisce in modo insicuro.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";
import { db } from "./db.js";

const app = express();
app.use(express.json());

app.get("/api/users", async (req, res) => {
  const email = req.query.email;
  const sql = `SELECT id, email, role FROM users WHERE email = '${email}'`;

  const result = await db.query(sql);
  return res.json(result.rows);
});
```

Frontend vulnerabile (React):


```
async function searchUser(email) {  
  const response = await fetch(`/api/users?email=${email}`, {  
    credentials: "include"  
  });  
  
  return response.json();  
}
```

Il frontend non codifica il parametro e il backend lo concatena nella query. Il problema decisivo rimane comunque lato server.

4. Scenario di attacco (solo descrizione teorica, non operativa)

Un utente invia un valore progettato per modificare il significato della query. Il database riceve una stringa SQL diversa da quella prevista dallo sviluppatore. In termini teorici, cio' potrebbe consentire lettura di record non autorizzati, bypass di controlli logici, modifica dei dati o cancellazione di informazioni.

5. Impatto sulla sicurezza del sistema

L'impatto puo' essere critico:

- Accesso non autorizzato a dati personali.
- Modifica o cancellazione di record.
- Bypass di autenticazione o autorizzazione.
- Esposizione di struttura del database.
- Compromissione dell'integrita' e della riservatezza dei dati.

6. Versione corretta del codice (mitigata)

Backend sicuro:

```
import express from "express";
import { z } from "zod";
import { db } from "./db.js";

const app = express();
app.use(express.json());

const searchSchema = z.object({
  email: z.string().trim().email().max(254)
});

app.get("/api/users", async (req, res) => {
  const parsed = searchSchema.safeParse(req.query);

  if (!parsed.success) {
    return res.status(400).json({ error: "Invalid query" });
  }

  const result = await db.query(
    "SELECT id, email, role FROM users WHERE email = $1",
    [parsed.data.email]
  );

  return res.json(result.rows);
});
```

Frontend sicuro:

```
async function searchUser(email) {
  const params = new URLSearchParams({ email });

  const response = await fetch(`/api/users?${params.toString()}`, {
    credentials: "include"
  });

  if (!response.ok) {
    throw new Error("User search failed");
  }

  return response.json();
}
```

7. Mitigazioni

Lato backend:

- Usare prepared statements o query parametrizzate.
- Usare ORM o query builder configurati correttamente.

- Validare input con `zod`, `joi` o `express-validator`.
- Evitare di esporre dettagli di errore SQL al client.
- Applicare il principio del privilegio minimo all'utente database.
- Separare autorizzazione applicativa da semplice presenza del dato nel database.

Lato frontend:

- Codificare parametri con `URLSearchParams`.
- Limitare campi e filtri inviati dall'utente a valori previsti dalla UI.
- Non considerare la validazione frontend come misura di sicurezza sufficiente.

Lato rete:

- Proteggere la connessione backend-database con canali cifrati quando attraversa reti non fidate.
- Isolare il database in una rete privata non esposta direttamente a Internet.
- Limitare accessi tramite firewall, security group e allowlist interne.

● Furto e manipolazione dei cookie

1. Descrizione tecnica

I cookie sono piccoli dati associati a un dominio e gestiti dal browser. Possono contenere identificativi di sessione, preferenze o token. Se configurati male, possono essere letti da JavaScript, trasmessi su HTTP, inviati in contesti cross-site non necessari o manipolati dall'utente.

Un errore comune e' inserire nei cookie dati sensibili o autorizzativi in chiaro, ad esempio `role=admin` o `userId=42`, e fidarsi di tali valori lato server. Un cookie deve essere considerato input controllabile dal client, a meno che non contenga solo un identificativo opaco verificato lato server oppure sia firmato e comunque validato.

2. Come si manifesta in React + Express

In React + Express il problema puo' manifestarsi quando:

- Express imposta cookie senza `HttpOnly`, `Secure` o `SameSite`.
- Il frontend legge e scrive manualmente cookie di autenticazione con `document.cookie`.
- Il backend memorizza ruoli o permessi direttamente nel cookie.
- Un token JWT e' salvato in cookie non protetto o in storage accessibile a JavaScript senza valutazione del rischio.
- L'applicazione non usa HTTPS in modo obbligatorio.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";

const app = express();
app.use(express.json());

app.post("/api/login", async (req, res) => {
  const user = await verifyPassword(req.body.email, req.body.password);

  if (!user) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  res.cookie("userId", String(user.id));
  res.cookie("role", user.role);

  return res.json({ ok: true });
});

app.get("/api/admin/reports", async (req, res) => {
  if (req.cookies.role !== "admin") {
    return res.status(403).json({ error: "Forbidden" });
  }

  return res.json(await getReports());
});
```

Frontend vulnerabile (React):

```
function saveUiSession(userId) {
  document.cookie = `userId=${userId}; path=/`;
}
```

Il cookie e' leggibile o modificabile lato client e viene usato come fonte di autorizzazione.

4. Scenario di attacco (solo descrizione teorica, non operativa)

Un soggetto non autorizzato ottiene accesso a cookie non protetti o modifica valori memorizzati nel browser. Se il server si fida dei valori del cookie per determinare identita' o ruolo, puo' attribuire privilegi errati alla richiesta. Se il cookie di sessione viene letto da codice client non autorizzato, puo' essere riutilizzato fino alla scadenza o alla revoca.

5. Impatto sulla sicurezza del sistema

L'impatto include:

- Impersonificazione dell'utente.
- Escalation di privilegi se ruoli e permessi sono manipolabili.
- Accesso a dati riservati.

- Persistenza di sessioni non autorizzate.
- Violazione dei principi di autenticazione e autorizzazione.

6. **Versione corretta del codice (mitigata)**

Backend sicuro:

```
import express from "express";
import session from "express-session";

const app = express();
app.set("trust proxy", 1);
app.use(express.json());

app.use(
  session({
    name: "__Host-sid",
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true,
      secure: true,
      sameSite: "lax",
      path: "/",
      maxAge: 1000 * 60 * 60
    }
  })
);

app.post("/api/login", async (req, res) => {
  const user = await verifyPassword(req.body.email, req.body.password);

  if (!user) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  req.session.userId = user.id;
  req.session.role = user.role;

  return res.json({ ok: true });
});

function requireAdmin(req, res, next) {
  if (!req.session.userId || req.session.role !== "admin") {
    return res.status(403).json({ error: "Forbidden" });
  }

  return next();
}

app.get("/api/admin/reports", requireAdmin, async (req, res) => {
  return res.json(await getReports());
});
```

Frontend sicuro:

```
async function login(email, password) {
  const response = await fetch("/api/login", {
    method: "POST",
    credentials: "include",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password })
  });

  if (!response.ok) {
    throw new Error("Login failed");
  }

  return response.json();
}
```

7. Mitigazioni

Lato backend:

- Usare cookie `HttpOnly` per impedire accesso tramite JavaScript.
- Usare cookie `Secure` per trasmetterli solo su HTTPS.
- Usare `SameSite=Lax` o `Strict` quando compatibile con il flusso applicativo.
- Usare prefisso `__Host-` per cookie di sessione quando possibile.
- Salvare lato client solo identificativi opachi, non ruoli o segreti.
- Verificare autorizzazione lato server a ogni richiesta.

Lato frontend:

- Non gestire manualmente cookie di sessione con `document.cookie`.
- Evitare di salvare JWT in `localStorage` se il rischio XSS non e' adeguatamente controllato.
- Considerare il frontend come ambiente non fidato: ogni controllo deve essere replicato lato backend.

Lato rete:

- Imporre HTTPS e HSTS.
- Evitare invio di cookie su sottodomini non necessari.
- Limitare dominio e path dei cookie.
- Monitorare configurazioni di reverse proxy e terminazione TLS.

● Session Hijacking

1. Descrizione tecnica

Il Session Hijacking consiste nell'uso non autorizzato di una sessione valida. La sessione puo' essere rappresentata da un cookie di sessione, da un JWT o da un altro token bearer. Un token bearer e' sufficiente a ottenere accesso finche' rimane valido: chi lo possiede puo' presentarlo al server.

Le cause frequenti sono session ID prevedibili, cookie non protetti, token troppo longevi, mancata rigenerazione della sessione al login, assenza di revoca, esposizione tramite XSS, trasmissione su HTTP o log contenenti token.

2. Come si manifesta in React + Express

In React + Express, il problema puo' manifestarsi quando:

- Express genera token deboli o prevedibili.
- La sessione non viene rigenerata dopo l'autenticazione.
- I cookie non hanno attributi di sicurezza.
- JWT con lunga scadenza sono salvati in storage accessibile a script.
- Logout e rotazione token non invalidano realmente la sessione.
- API e frontend accettano sessioni anche dopo cambio password o revoca account.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";

const app = express();
app.use(express.json());

const sessions = new Map();

app.post("/api/login", async (req, res) => {
  const user = await verifyPassword(req.body.email, req.body.password);

  if (!user) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  const sessionId = `${user.id}-${Date.now()}`;
  sessions.set(sessionId, { userId: user.id });

  res.cookie("sid", sessionId);
  return res.json({ ok: true });
});
```

Frontend vulnerabile (React):

```
async function loadProfile() {  
  const token = localStorage.getItem("accessToken");  
  
  const response = await fetch("/api/profile", {  
    headers: { Authorization: `Bearer ${token}` }  
  });  
  
  return response.json();  
}
```

L'esempio mostra due pattern rischiosi: session ID prevedibile e token persistente in storage accessibile a script.

4. Scenario di attacco (solo descrizione teorica, non operativa)

Un identificativo di sessione o token viene ottenuto da un soggetto non autorizzato tramite esposizione client-side, log, trasmissione non protetta o vulnerabilit  applicativa. Il token viene poi presentato al server come se fosse dell'utente legittimo. Il server, non distinguendo il possessore reale del token, concede accesso fino a scadenza o revoca.

5. Impatto sulla sicurezza del sistema

L'impatto   alto:

- Accesso completo all'account fino alla scadenza della sessione.
- Modifica di dati e impostazioni.
- Azioni eseguite a nome dell'utente.
- Persistenza se refresh token o sessioni lunghe non vengono revocati.
- Rischio amplificato per account amministrativi.

6. Versione corretta del codice (mitigata)

Backend sicuro:

```
import express from "express";
import session from "express-session";

const app = express();
app.set("trust proxy", 1);
app.use(express.json());

app.use(
  session({
    name: "__Host-sid",
    secret: process.env.SESSION_SECRET,
    resave: false,
    saveUninitialized: false,
    rolling: true,
    cookie: {
      httpOnly: true,
      secure: true,
      sameSite: "lax",
      path: "/",
      maxAge: 1000 * 60 * 30
    }
  })
);

app.post("/api/login", async (req, res, next) => {
  const user = await verifyPassword(req.body.email, req.body.password);

  if (!user) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  req.session.regenerate((error) => {
    if (error) return next(error);

    req.session.userId = user.id;
    req.session.authenticatedAt = Date.now();

    return res.json({ ok: true });
  });
});

app.post("/api/logout", (req, res, next) => {
  req.session.destroy((error) => {
    if (error) return next(error);

    res.clearCookie("__Host-sid", { path: "/" });
    return res.status(204).send();
  });
});
```



```
async function loadProfile() {  
  const response = await fetch("/api/profile", {  
    credentials: "include"  
  });  
  
  if (!response.ok) {  
    throw new Error("Profile request failed");  
  }  
  
  return response.json();  
}
```

7. Mitigazioni

Lato backend:

- Usare generatori crittograficamente sicuri per identificativi di sessione.
- Rigenerare la sessione dopo login e cambi di privilegio.
- Impostare timeout di inattività e scadenza assoluta.
- Invalidare sessioni a logout, cambio password e revoca account.
- Non loggare cookie, token o header **Authorization**.
- Applicare rate limiting su login e refresh token.

Lato frontend:

- Preferire cookie **HttpOnly** per sessioni web tradizionali.
- Se si usano JWT, mantenere access token brevi e refresh token protetti.
- Non esporre token in URL, query string o log client-side.
- Gestire logout in modo che lo stato locale sia cancellato.

Lato rete:

- Usare HTTPS obbligatorio.
- Impedire caching improprio di risposte sensibili.
- Applicare HSTS e controllare la configurazione TLS.
- Evitare trasmissione di credenziali su reti o proxy non fidati.

● Clickjacking

1. Descrizione tecnica

Il Clickjacking e' una tecnica in cui una pagina legittima viene incorporata in un frame o iframe controllato da un altro sito. L'utente puo' essere indotto a cliccare su elementi dell'applicazione senza comprenderne il reale contesto. La vulnerabilita' nasce quando l'applicazione permette di essere renderizzata all'interno di frame di origini non autorizzate.

La protezione si basa principalmente su header HTTP interpretati dal browser: **Content-Security-Policy** con direttiva **frame-ancestors** e, per compatibilita', **X-Frame-Options**.

2. Come si manifesta in React + Express

In React + Express, il problema puo' manifestarsi quando:

- Express non imposta header anti-framing.
- La SPA espone pagine con azioni sensibili e pulsanti confermabili con un solo click.
- Il frontend non richiede conferme robuste per operazioni critiche.
- L'applicazione deve essere embeddabile ma non limita le origini autorizzate.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";

const app = express();
app.use(express.json());

app.post("/api/account/delete", async (req, res) => {
  await deleteAccount(req.session.userId);
  return res.json({ ok: true });
});
```

Frontend vulnerabile (React):

```
export function DangerZone() {  
  async function deleteAccount() {  
    await fetch("/api/account/delete", {  
      method: "POST",  
      credentials: "include"  
    });  
  }  
  
  return <button onClick={deleteAccount}>Delete account</button>;  
}
```

La UI espone un'azione sensibile e il server non comunica al browser alcuna restrizione sull'incorporamento in frame.

4. Scenario di attacco (solo descrizione teorica, non operativa)

Una pagina esterna incorpora l'applicazione in un frame e sovrappone elementi grafici per alterare la percezione dell'utente. L'utente crede di interagire con la pagina esterna, ma il click viene indirizzato all'applicazione autenticata. Il problema è mitigato impedendo al browser di renderizzare l'applicazione in contesti non autorizzati.

5. Impatto sulla sicurezza del sistema

L'impatto può includere:

- Conferma involontaria di operazioni sensibili.
- Modifica di impostazioni dell'account.
- Interazioni non intenzionali con pannelli amministrativi.
- Abuso di sessioni già autenticate.

6. Versione corretta del codice (mitigata)

Backend sicuro:

```
import express from "express";
import helmet from "helmet";

const app = express();
app.use(express.json());

app.use(
  helmet({
    xFrameOptions: { action: "deny" },
    contentSecurityPolicy: {
      useDefaults: true,
      directives: {
        "default-src": ["'self'"],
        "frame-ancestors": ["'none'"]
      }
    }
  })
);

app.post("/api/account/delete", requireAuth, requireCsrft, async (req, res) => {
  await deleteAccount(req.session.userId);
  return res.json({ ok: true });
});
```

Frontend sicuro:


```
import { useState } from "react";

export function DangerZone({ csrfToken }) {
  const [confirmation, setConfirmation] = useState("");

  async function deleteAccount() {
    if (confirmation !== "DELETE") {
      return;
    }

    const response = await fetch("/api/account/delete", {
      method: "POST",
      credentials: "include",
      headers: { "X-CSRF-Token": csrfToken }
    });

    if (!response.ok) {
      throw new Error("Account deletion failed");
    }
  }

  return (
    <form onSubmit={(event) => event.preventDefault()}>
      <label htmlFor="delete-confirmation">Confirm deletion</label>
      <input
        id="delete-confirmation"
        value={confirmation}
        onChange={(event) => setConfirmation(event.target.value)}
      />
      <button type="button" onClick={deleteAccount}>
        Delete account
      </button>
    </form>
  );
}
```

7. Mitigazioni

Lato backend:

- Impostare **Content-Security-Policy: frame-ancestors 'none'** o origini specifiche.
- Impostare **X-Frame-Options: DENY** o **SAMEORIGIN** per compatibilit .
- Richiedere token anti-CSRF per operazioni state-changing.
- Richiedere riautenticazione o conferma esplicita per operazioni critiche.

Lato frontend:

- Evitare azioni distruttive confermabili con un singolo click.
- Usare conferme esplicite e stati intermedi per operazioni irreversibili.

- Non basare la protezione su script anti-frame lato client, facilmente aggirabili o non affidabili quanto gli header.

Lato rete:

- Assicurarsi che proxy e CDN non rimuovano header di sicurezza.
- Verificare gli header esposti in produzione, non solo in sviluppo.
- Usare HTTPS per evitare alterazioni degli header in transito.

● Man-in-the-Middle (MITM)

1. Descrizione tecnica

Un attacco Man-in-the-Middle e' una condizione in cui un soggetto intermedio riesce a osservare, intercettare o alterare la comunicazione tra client e server. L'analisi deve distinguere tra HTTP e HTTPS.

HTTP e' un protocollo applicativo in chiaro: richieste e risposte possono essere lette da chi controlla un punto della rete attraversata dal traffico. Cookie, URL, payload JSON e header non sono cifrati. HTTP non garantisce riservatezza, integrita' o autenticita' del server.

HTTPS e' HTTP trasportato dentro TLS. TLS fornisce:

- **Autenticazione del server** tramite certificato digitale emesso da una Certificate Authority riconosciuta dal client.
- **Riservatezza** tramite cifratura simmetrica del traffico applicativo.
- **Integrita'** tramite meccanismi crittografici che rilevano alterazioni.
- **Protezione dal downgrade** se combinato con HSTS e configurazioni moderne.

Il TLS handshake, ad alto livello, avviene così:

1. Il client avvia la connessione e comunica parametri supportati, come versioni TLS e suite crittografiche.
2. Il server presenta il certificato e i parametri scelti.
3. Il browser verifica che il certificato sia valido, non scaduto, emesso per il dominio richiesto e firmato da una CA attendibile.
4. Client e server stabiliscono chiavi di sessione tramite uno scambio crittografico.
5. Il traffico HTTP successivo viene cifrato e autenticato all'interno del canale TLS.

Il packet sniffing su reti non sicure, come Wi-Fi pubblici o reti locali non controllate, consente di osservare traffico in chiaro. Con HTTPS correttamente configurato, un osservatore può ancora vedere metadati di rete come indirizzi IP, orari e dimensioni approssimative, ma non il contenuto applicativo.

2. Come si manifesta in React + Express

In React + Express, MITM può diventare rilevante quando:

- La SPA chiama API con URL `http://` invece di `https://`.
- Il backend non forza redirect da HTTP a HTTPS.
- I cookie non hanno attributo `Secure`.
- HSTS non è configurato.
- TLS è terminato su proxy/CDN ma il traffico interno verso Express usa canali non protetti in una rete non fidata.
- La configurazione CORS permette origini non controllate.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";

const app = express();
app.use(express.json());

app.post("/api/login", async (req, res) => {
  const user = await verifyPassword(req.body.email, req.body.password);

  if (!user) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  res.cookie("sid", createSession(user.id), {
    httpOnly: true
  });

  return res.json({ ok: true });
});

app.listen(3000);
```

Frontend vulnerabile (React):

```
const API_BASE_URL = "http://api.example.test";

export async function login(email, password) {
  const response = await fetch(`${API_BASE_URL}/api/login`, {
    method: "POST",
    credentials: "include",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password })
  });

  return response.json();
}
```

4. Scenario di attacco (solo descrizione teorica, non operativa)

Su una rete non fidata, la comunicazione HTTP puo' essere osservata o alterata da un soggetto intermedio. Dati di login, cookie o risposte API possono essere letti durante il transito. In assenza di integrita' del trasporto, la risposta potrebbe anche essere modificata prima di arrivare al browser. HTTPS correttamente configurato impedisce che il contenuto applicativo sia leggibile o alterabile senza rilevamento.

5. Impatto sulla sicurezza del sistema

L'impatto comprende:

- Esposizione di credenziali.

- Furto di cookie o token.
- Alterazione di risposte API.
- Iniezione di codice nella SPA se asset statici sono serviti su HTTP.
- Perdita di integrita' delle operazioni client-server.

6. **Versione corretta del codice (mitigata)**

Backend sicuro:

```
import express from "express";
import helmet from "helmet";

const app = express();
app.set("trust proxy", 1);
app.use(express.json());

app.use((req, res, next) => {
  if (req.secure || req.hostname === "localhost") {
    return next();
  }

  return res.redirect(301, `https://${req.headers.host}${req.originalUrl}`);
});

app.use(
  helmet({
    strictTransportSecurity: {
      maxAge: 31536000,
      includeSubDomains: true,
      preload: true
    }
  })
);

app.post("/api/login", async (req, res) => {
  const user = await verifyPassword(req.body.email, req.body.password);

  if (!user) {
    return res.status(401).json({ error: "Invalid credentials" });
  }

  res.cookie("__Host-sid", createSession(user.id), {
    httpOnly: true,
    secure: true,
    sameSite: "lax",
    path: "/",
    maxAge: 1000 * 60 * 60
  });

  return res.json({ ok: true });
});
```

Frontend sicuro:

```
export async function login(email, password) {
  const response = await fetch("/api/login", {
    method: "POST",
    credentials: "include",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ email, password })
  });

  if (!response.ok) {
    throw new Error("Login failed");
  }

  return response.json();
}
```

L'uso di URL relativi riduce errori di configurazione quando frontend e API sono serviti sotto lo stesso origin HTTPS o tramite reverse proxy.

7. Mitigazioni

Lato backend:

- Forzare HTTPS in produzione.
- Configurare HSTS con durata adeguata.
- Usare cookie `Secure`, `HttpOnly` e `SameSite`.
- Verificare la configurazione del reverse proxy con `app.set("trust proxy", 1)`.
- Non accettare credenziali su endpoint HTTP.
- Usare configurazioni TLS moderne a livello di proxy, load balancer o web server.

Lato frontend:

- Usare URL relativi o endpoint HTTPS espliciti.
- Evitare mixed content: nessuna risorsa HTTP dentro pagine HTTPS.
- Non ignorare errori TLS in ambienti non controllati.
- Gestire in modo sicuro errori di rete e sessione.

Lato rete:

- Usare TLS end-to-end o cifratura anche tra proxy e backend quando la rete interna non e' pienamente fidata.
- Separare reti pubbliche, private e amministrative.
- Monitorare scadenza e validita' dei certificati.
- Preferire TLS 1.2 o TLS 1.3 con suite crittografiche moderne.
- Ridurre esposizione di servizi interni tramite firewall e security group.

● Attacchi a livello rete: DNS spoofing e ARP spoofing

1. Descrizione tecnica

Gli attacchi a livello rete mirano a deviare, osservare o alterare il traffico prima che raggiunga il server legittimo. In questa relazione vengono considerati in modo concettuale due scenari:

- **DNS spoofing:** alterazione della risoluzione del nome di dominio, in modo che il client riceva un indirizzo IP non corretto per un dominio legittimo.
- **ARP spoofing:** manipolazione concettuale del mapping tra indirizzo IP e indirizzo MAC in una rete locale IPv4, con possibile deviazione del traffico verso un nodo intermedio.

Questi scenari non sono vulnerabilit  specifiche di React o Express, ma incidono sulla sicurezza della comunicazione browser-server. TLS correttamente validato e' il controllo principale per impedire che una deviazione di traffico si trasformi in compromissione del contenuto applicativo.

2. Come si manifesta in React + Express

In un'applicazione React + Express, il rischio aumenta quando:

- L'applicazione e' accessibile anche via HTTP.
- Il frontend usa endpoint API hardcoded non protetti.
- Il browser non riceve HSTS.
- Gli utenti ignorano errori di certificato.
- API, pannelli amministrativi o ambienti di staging sono esposti su reti non protette.
- Il backend accetta richieste da origini non previste.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import cors from "cors";
import express from "express";

const app = express();
app.use(express.json());

app.use(
  cors({
    origin: true,
    credentials: true
  })
);

app.get("/api/session", (req, res) => {
  return res.json({ userId: req.session.userId });
});
```

Frontend vulnerabile (React):


```
const API_BASE_URL = "http://192.0.2.10:3000";

export async function getSession() {
  const response = await fetch(`${API_BASE_URL}/api/session`, {
    credentials: "include"
  });

  return response.json();
}
```

L'esempio evidenzia due criticita': uso di HTTP e CORS eccessivamente permissivo.

4. Scenario di attacco (solo descrizione teorica, non operativa)

In uno scenario DNS spoofing, il client potrebbe essere indirizzato verso un host non legittimo. In uno scenario ARP spoofing, il traffico in una rete locale potrebbe passare da un nodo intermedio. Se l'applicazione usa HTTP, il contenuto e' leggibile e modificabile. Se usa HTTPS con certificato valido e HSTS, il browser rileva l'assenza di un certificato coerente per il dominio e blocca o segnala la connessione.

5. Impatto sulla sicurezza del sistema

L'impatto potenziale include:

- Reindirizzamento verso servizi non legittimi.
- Packet sniffing su traffico HTTP.
- Alterazione di risposte o asset statici non cifrati.
- Furto di credenziali se inserite su canali non validati.
- Interruzione o degradazione del servizio.

6. Versione corretta del codice (mitigata)

Backend sicuro:

```
import cors from "cors";
import express from "express";
import helmet from "helmet";

const app = express();
app.set("trust proxy", 1);
app.use(express.json());

const allowedOrigins = new Set([
  "https://app.example.com",
  "https://admin.example.com"
]);

app.use(
  cors({
    origin(origin, callback) {
      if (!origin || allowedOrigins.has(origin)) {
        return callback(null, true);
      }

      return callback(new Error("Origin not allowed"));
    },
    credentials: true
  })
);

app.use(
  helmet({
    strictTransportSecurity: {
      maxAge: 31536000,
      includeSubDomains: true
    }
  })
);

app.get("/api/session", requireAuth, (req, res) => {
  return res.json({ userId: req.session.userId });
});
```

Frontend sicuro:

```
export async function getSession() {  
  const response = await fetch("/api/session", {  
    credentials: "include"  
  });  
  
  if (!response.ok) {  
    throw new Error("Session request failed");  
  }  
  
  return response.json();  
}
```

7. Mitigazioni

Lato backend:

- Limitare CORS a origin espliciti.
- Richiedere autenticazione e autorizzazione anche se la rete e' considerata interna.
- Non esporre pannelli amministrativi su reti pubbliche senza ulteriori controlli.
- Applicare logging e alert su host header anomali, errori TLS e pattern di accesso insoliti.

Lato frontend:

- Usare esclusivamente endpoint HTTPS.
- Evitare IP hardcoded o URL HTTP in bundle di produzione.
- Non implementare logiche che invitano l'utente a ignorare errori TLS.
- Mostrare stati di errore sicuri quando la sessione non puo' essere verificata.

Lato rete:

- Usare HTTPS con certificati validi per tutti i domini applicativi.
- Abilitare HSTS sui domini principali.
- Considerare DNSSEC dove supportato e appropriato.
- Proteggere reti locali con segmentazione, switch gestiti e controlli anti-spoofing.
- Limitare accesso a database e backend tramite reti private, firewall e VPN amministrative.

● Vulnerabilita' HTTP/TLS e mancanza di header di sicurezza

1. Descrizione tecnica

HTTP Security Headers sono intestazioni inviate dal server per istruire il browser su come trattare la pagina, le risorse e il contesto di esecuzione. Non sostituiscono la correzione delle vulnerabilita' applicative, ma riducono la superficie di attacco e limitano l'impatto di errori.

Gli header principali sono:

- **Content-Security-Policy (CSP):** definisce da quali origini e' possibile caricare script, stili, immagini, font, frame e altre risorse. Aiuta a mitigare XSS e data injection.
- **Strict-Transport-Security (HSTS):** indica al browser di usare HTTPS per il dominio per un certo periodo, riducendo rischi di downgrade verso HTTP.
- **X-Frame-Options:** impedisce o limita l'incorporamento della pagina in frame, utile contro clickjacking. La direttiva CSP `frame-ancestors` e' piu' moderna.
- **X-Content-Type-Options:** con valore `nosniff`, impedisce al browser di interpretare una risorsa con tipo diverso da quello dichiarato.
- **Referrer-Policy:** controlla quali informazioni dell'URL sorgente vengono inviate tramite header `Referer`.
- **Permissions-Policy:** limita l'uso di funzionalita' browser come geolocalizzazione, camera, microfono e fullscreen.

2. Come si manifesta in React + Express

In React + Express, la mancanza di header si manifesta quando:

- Express serve API e static files senza `helmet`.
- La SPA consente script inline o risorse da origini non controllate.
- Il dominio non forza HTTPS con HSTS.
- L'applicazione puo' essere inclusa in iframe di terzi.
- Il browser puo' fare MIME sniffing di risorse caricate.
- Informazioni sensibili possono finire nel referrer verso siti esterni.

3. Esempio di codice vulnerabile

Backend vulnerabile (Express.js):

```
import express from "express";

const app = express();
app.use(express.json());
app.use(express.static("dist"));

app.get("/api/health", (req, res) => {
  return res.json({ ok: true });
});
```

Frontend vulnerabile (React):


```
export function ExternalLink({ url, children }) {  
  return (  
    <a href={url} target="_blank">  
      {children}  
    </a>  
  );  
}
```

Il backend non invia header difensivi. Il link esterno manca di attributi che separano il nuovo contesto di navigazione dalla pagina originale.

4. Scenario di attacco (solo descrizione teorica, non operativa)

In assenza di header, il browser applica policy predefinite meno restrittive. Un errore XSS puo' avere impatto maggiore se la CSP consente script da origini arbitrarie. Un'applicazione senza HSTS puo' essere caricata inizialmente su HTTP. Una pagina senza protezione anti-frame puo' essere incorporata in contesti non autorizzati. Un referrer troppo dettagliato puo' esporre URL interni o parametri sensibili verso domini esterni.

5. Impatto sulla sicurezza del sistema

L'impatto include:

- Maggiore probabilita' o impatto di XSS.
- Rischio di clickjacking.
- Downgrade o primo accesso su HTTP in assenza di HSTS.
- Esposizione non necessaria di metadati tramite referrer.
- Uso non controllato di funzionalita' browser sensibili.
- Interpretazione errata di risorse a causa di MIME sniffing.

6. Versione corretta del codice (mitigata)

Backend sicuro con `helmet`:

```
import express from "express";
import helmet from "helmet";
import rateLimit from "express-rate-limit";

const app = express();
app.set("trust proxy", 1);
app.use(express.json({ limit: "1mb" }));

app.use(
  helmet({
    contentSecurityPolicy: {
      useDefaults: true,
      directives: {
        "default-src": ["'self'"],
        "script-src": ["'self'"],
        "style-src": ["'self'"],
        "img-src": ["'self'", "data:", "https:"],
        "font-src": ["'self'"],
        "connect-src": ["'self'", "https://api.example.com"],
        "object-src": ["'none'"],
        "base-uri": ["'self'"],
        "frame-ancestors": ["'none'"],
        "form-action": ["'self'"],
        "upgrade-insecure-requests": []
      }
    },
    strictTransportSecurity: {
      maxAge: 31536000,
      includeSubDomains: true,
      preload: true
    },
    xFrameOptions: {
      action: "deny"
    },
    xContentTypeOptions: true,
    referrerPolicy: {
      policy: "strict-origin-when-cross-origin"
    }
  })
);

app.use((req, res, next) => {
  res.setHeader(
    "Permissions-Policy",
    "camera=(), microphone=(), geolocation=(), payment=(), usb=()"
  );
  next();
});

app.use(
  "/api",
  rateLimit({
```

```
    windowMs: 15 * 60 * 1000,  
    max: 300,  
    standardHeaders: true,  
    legacyHeaders: false  
  })  
);  
  
app.use(express.static("dist", {  
  index: false,  
  maxAge: "1y",  
  immutable: true  
}));  
  
app.get("/api/health", (req, res) => {  
  return res.json({ ok: true });  
});
```

Frontend sicuro:


```
export function ExternalLink({ url, children }) {  
  return (  
    <a href={url} target="_blank" rel="noreferrer noopener">  
      {children}  
    </a>  
  );  
}
```

7. Mitigazioni

Lato backend:

- Usare `helmet` come baseline.
- Definire CSP compatibile con il bundle React senza aprire `script-src` a origini non necessarie.
- Abilitare HSTS solo quando HTTPS e' correttamente funzionante su dominio e sottodomini interessati.
- Applicare rate limiting agli endpoint API.
- Limitare dimensione dei payload JSON.
- Validare tutti gli input e usare prepared statements o ORM sicuri.

Lato frontend:

- Evitare dipendenze esterne non necessarie caricate da CDN non controllate.
- Evitare script inline se si vuole mantenere una CSP forte.
- Usare `rel="noopener noreferrer"` nei link con `target="_blank"`.
- Non inserire dati sensibili negli URL.

Lato rete:

- Configurare TLS su proxy, load balancer o server edge con protocolli moderni.
- Forzare redirect HTTP -> HTTPS.
- Verificare periodicamente certificati, catena di trust e scadenze.
- Evitare mixed content.
- Proteggere il canale tra reverse proxy e backend se attraversa reti condivise.

4. Conclusioni

L'analisi mostra che una web application React + Express deve essere protetta su piu' livelli. Le vulnerabilita' applicative, come XSS, CSRF e SQL Injection, derivano spesso da input non validato, output non controllato o mancata separazione tra dati e codice. Le vulnerabilita' di sessione, come furto di cookie e Session Hijacking, dipendono invece dalla gestione di credenziali, cookie, token, durata delle sessioni e revoca. Le vulnerabilita' di rete e trasporto, come MITM, DNS spoofing concettuale e ARP spoofing concettuale, evidenziano che la sicurezza del codice non e' sufficiente se la comunicazione browser-server non e' protetta da HTTPS, TLS, HSTS e cookie configurati correttamente.

Nel frontend React, l'escaping automatico offre una protezione importante contro XSS, ma puo' essere indebolito da `dangerouslySetInnerHTML`, storage insicuro dei token o dipendenze non controllate. Nel backend Express, la sicurezza deve essere applicata con middleware e controlli espliciti: `helmet`, rate limiting,

validazione con `zod`, `joi` o `express-validator`, query parametrizzate, sessioni robuste e controlli di autorizzazione lato server.

La sicurezza della rete completa il modello difensivo: HTTPS deve essere obbligatorio, TLS deve essere configurato correttamente, i cookie devono usare `Secure`, `HttpOnly` e `SameSite`, e gli header HTTP devono guidare il comportamento del browser. In una SPA, il browser non e' solo un visualizzatore, ma un componente attivo dell'architettura di sicurezza.

Il principio fondamentale e' la **security by design**: ogni funzionalita' deve essere progettata considerando minacce, trust boundary, dati trattati e impatto di compromissione. Questo approccio e' coerente con il riferimento concettuale dell'**OWASP Top 10**, che classifica rischi come broken access control, injection, security misconfiguration, vulnerable components, identification and authentication failures e server-side request forgery. Una progettazione sicura non elimina completamente il rischio, ma lo riduce tramite controlli coerenti, verificabili e distribuiti tra frontend, backend, browser e rete.